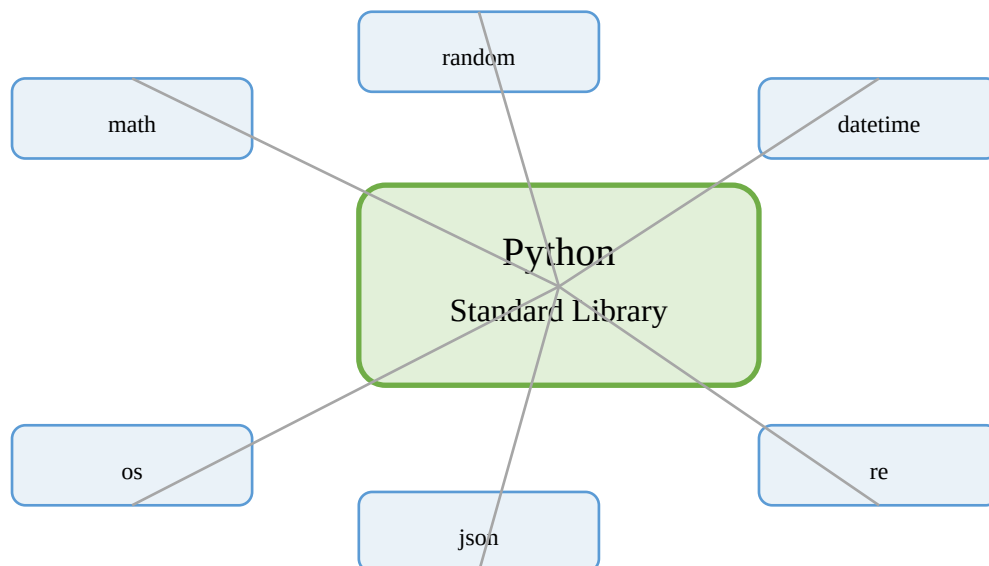


PYTHON MODULES & PACKAGES

DAY 9 • Complete Study Notes

Modules • Importing • Custom Modules • Packages • Standard Library



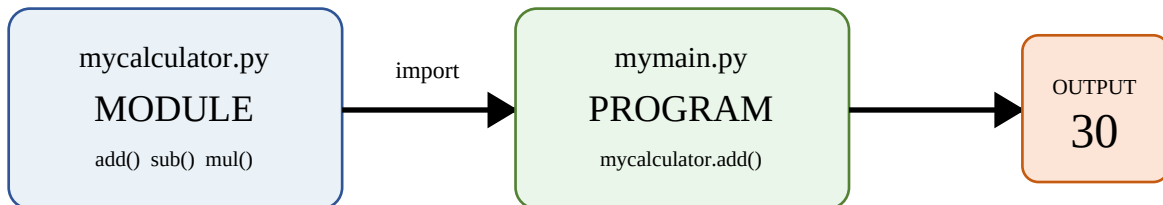
Goal: Understand how Python code is divided, reused, imported, and organized.

1. What is a Module?

Definition: A module is a Python file (.py) that contains reusable code such as functions, variables, classes, and statements.

Why modules? They make programs reusable, organized, readable, and easier to maintain.

Think of a module as a **toolbox**: you keep useful tools in one file and use them whenever required.



Key idea: One Python file → one module.

2. Importing Modules

The **import** keyword makes another module available in the current program.

A. Import complete module

```
import math
print(math.sqrt(25))
```

B. Import a specific function

```
from math import sqrt
print(sqrt(25))
```

C. Import multiple functions

```
from math import sqrt, factorial
```

D. Import with an alias

```
import math as m
print(m.sqrt(25))
```

E. Import everything

```
from math import *
```

Best practice: Prefer clear imports such as **import math** or **from math import sqrt**. Avoid ***** in larger programs because names can become unclear or conflict.

3. Creating a Custom Module

A **custom module** is a Python file created by the programmer for project-specific reusable code.

Project structure:

```
pythonModule/  
■■■ mycalculator.py  
■■■ mymain.py
```

mycalculator.py

```
def add(n1, n2):  
    return n1 + n2  
  
def sub(n1, n2):  
    return n1 - n2  
  
def mul(n1, n2):  
    return n1 * n2
```

mymain.py

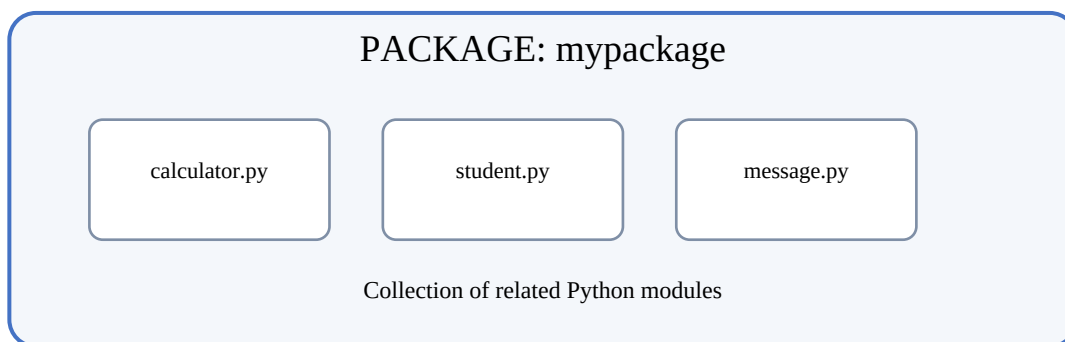
```
import mycalculator  
  
print(mycalculator.add(10, 20))  
print(mycalculator.sub(20, 10))  
print(mycalculator.mul(15, 3))
```

Output: **30 10 45**

Important: mycalculator.add() means: *module_name.function_name()*.

4. What is a Package?

A package is a directory used to organize related Python modules.



```
mypackage/  
■■■ __init__.py  
■■■ calculator.py  
■■■ student.py  
■■■ message.py
```

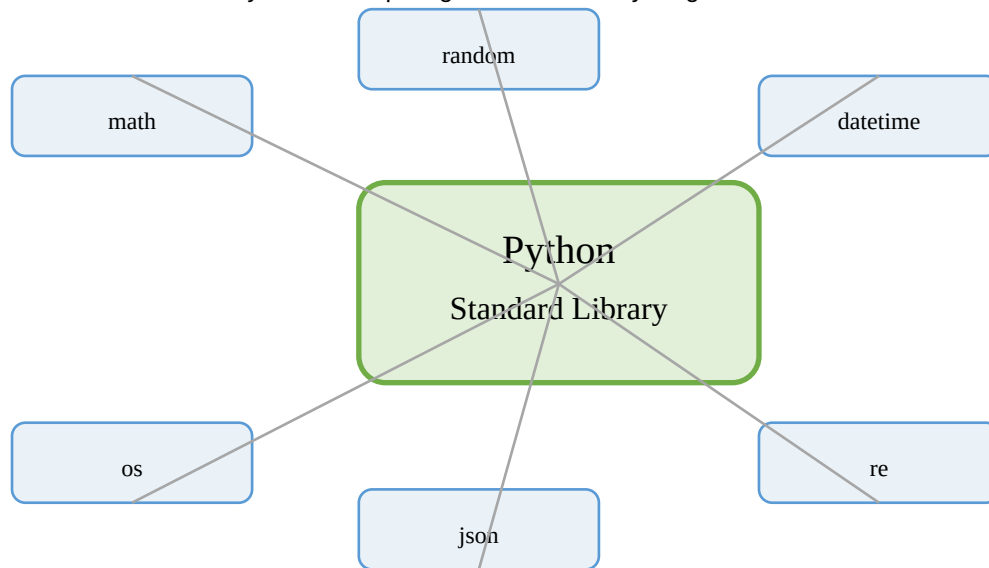
Example import:

```
from mypackage.calculator import add  
print(add(10, 20))
```

Benefits: better project organization, separation of related code, easier maintenance, and scalability.

5. Python Standard Library

Python comes with a large collection of ready-to-use modules called the **Python Standard Library**. These modules provide common functionality without requiring us to write everything from scratch.



Core modules for Day 9: math, random, datetime, calendar, os, sys, json, re, statistics, collections, time.

6. math Module

Used for mathematical functions and constants.

```
import math

print(math.sqrt(25)) # 5.0
print(math.pow(2, 3)) # 8.0
print(math.factorial(5)) # 120
print(math.ceil(4.2)) # 5
print(math.floor(4.8)) # 4
print(math.pi) # PI
```

Important methods:

sqrt() → square root | pow() → power | factorial() → factorial
ceil() → upper integer | floor() → lower integer | pi → π

Common mistake: math.add(), math.sub(), math.max(), math.min() are not math module functions. max(), min(), and sum() are Python built-ins.

7. random Module

Used to generate random values and select random items.

```
import random

names = ["Arun", "Rahul", "Amit", "Rohit", "Vivek"]

print(random.choice(names))
print(random.randint(1, 100))
print(random.randrange(1, 10))
print(random.random())
print(random.uniform(10, 20))
```

choice() — selects one random item.

```
print(random.choice(names))
```

shuffle() — randomly changes the order of a list.

```
random.shuffle(names)
print(names)
```

sample() — selects multiple unique random items.

```
print(random.sample(names, 2))
```

Quick difference: choice → 1 item | randint → integer | randrange → integer from range | shuffle → changes list order | sample → multiple unique items | random → 0 to 1 float | uniform → float in a range.

8. datetime Module

Used to work with dates and times.

```
import datetime

print(datetime.datetime.now())
print(datetime.date.today())
```

Useful objects: datetime, date, time, timedelta.

9. calendar Module

Used to display and work with calendars.

```
import calendar

print(calendar.month(2026, 9))
print(calendar.isleap(2024))
```

Useful functions: month(), calendar(), isleap().

10. os Module

Used to interact with the operating system, files, folders, and paths.

```
import os

print(os.getcwd())
print(os.listdir())
os.mkdir("test")
```

Common uses: current directory, listing files, creating folders, checking paths.

11. sys Module

Provides access to Python interpreter and system-related information.

```
import sys

print(sys.version)
print(sys.platform)
```

12. json Module

Used to work with JSON data, especially when exchanging data with APIs and applications.

```
import json

data = '{"name": "Arun", "age": 23}'
result = json.loads(data)
print(result["name"])

obj = {"name": "Arun", "age": 23}
print(json.dumps(obj))
```

Remember: loads() → JSON string to Python object | dumps() → Python object to JSON string.

13. re Module

Used for Regular Expressions (pattern matching in text).

```
import re

text = "My phone number is 9876543210"
result = re.search(r"\d+", text)
```

```
if result:
    print(result.group())
```

Uses: searching patterns, validation, finding numbers/emails, and replacing text patterns.

14. statistics Module

Used for common statistical calculations.

```
import statistics

marks = [70, 80, 90, 60, 100]
print(statistics.mean(marks))
print(statistics.median(marks))
```

Important: mean() → average | median() → middle value | mode() → most frequent value.

15. collections Module

Provides specialized container data types.

```
from collections import Counter

names = ["Arun", "Amit", "Arun", "Rahul", "Amit", "Arun"]
print(Counter(names))
```

Output: Counter({'Arun': 3, 'Amit': 2, 'Rahul': 1})

Important classes: Counter, defaultdict, deque, namedtuple.

16. time Module

Provides time-related functions.

```
import time

print("Hello")
time.sleep(2)
print("Python")
```

sleep(2) pauses program execution for 2 seconds.

17. Module Quick Reference

Module	Main Purpose	Example
math	Mathematical operations	math.sqrt(25)
random	Random values/items	random.choice(names)
datetime	Date and time	datetime.date.today()
calendar	Calendar operations	calendar.month()
os	OS, files, folders	os.getcwd()
sys	Interpreter/system info	sys.version
json	JSON data	json.loads()
re	Pattern matching	re.search()
statistics	Statistics	statistics.mean()
collections	Special containers	Counter()
time	Time operations	time.sleep(2)

18. Module vs Package

MODULE → One .py file
PACKAGE → Directory containing related modules

Example: calculator.py is a module. A folder containing calculator.py, student.py and message.py is a package.

19. Important Viva / Interview Questions

Q1. What is a module?

A Python file containing reusable code such as functions, variables, classes, and statements.

Q2. Why are modules used?

For code reuse, organization, readability, and easier maintenance.

Q3. How do you import a module?

```
import math
```

Q4. What is a custom module?

A module created by the programmer for project-specific reusable functionality.

Q5. What is a package?

A directory used to organize related Python modules.

Q6. What is the Standard Library?

A collection of ready-to-use modules provided with Python.

Q7. choice() vs sample()?

choice() selects one item; sample() selects multiple unique items.

Q8. randint() vs randrange()?

randint() includes both endpoints; randrange() excludes the stop value.

20. Day 9 Practical Structure

```
Day_9_Modules_packages/  
■■■ pythonModule/
```

```
#### mycalculator.py
#### mymain.py
#### mathmodule.py
#### randommodule.py
#### datetimemodule.py
#### calendarmodule.py
#### osmodule.py
#### sysmodule.py
#### jsonmodule.py
#### remodule.py
#### statisticmodule.py
#### collectionsmodule.py
#### timemodule.py
```

FINAL TAKEAWAY

Module → single reusable Python file.

Package → organized collection of related modules.

import → brings a module/function into your program.

Custom module → created by the programmer.

Standard Library → ready-made Python functionality.

Day 9 Core Practice: custom module + math + random + datetime + os + sys + json.